Greedy Problem Solving 3

- Gaurish Baliga

Problem 1: Candy Box

Problem Link: https://codeforces.com/problemset/problem/1183/D

There are n candies in a candy box. The type of the i-th candy is a_i ($1 \le a_i \le n$).

You have to prepare a gift using some of these candies with the following restriction: the numbers of candies of each type presented in a gift should be all distinct (i.e. for example, a gift having two candies of type 1 and two candies of type 2 is bad).

It is possible that multiple types of candies are completely absent from the gift. It is also possible that **not all candies** of some types will be taken to a gift.

Your task is to find out the **maximum** possible size of the single gift you can prepare using the candies you have.

You have to answer q independent queries.

Problem 1: Candy Box

Example

input

```
3
8
1 4 8 4 5 6 3 8
16
2 1 3 3 4 3 4 4 1 3 2 2 2 4 1 1
9
2 2 4 4 4 7 7 7 7
```

output

```
3
10
9
```

Note

In the first query, you can prepare a gift with two candies of type 8 and one candy of type 5, totalling to 3 candies.

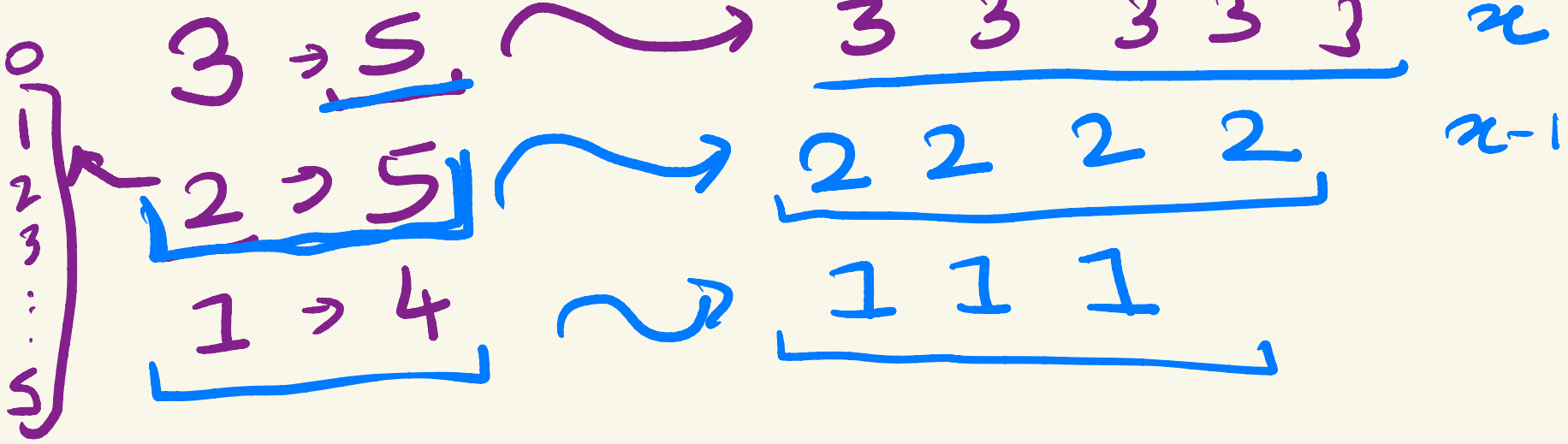
Note that this is not the only possible solution — taking two candies of type 4 and one candy of type 6 is also valid.

array a_i, $1 \le a_i \le n$

$\{t_1, f_1, t_2, f_2\}$ $f_i \neq f_j$

$\sum f_i$

Highest frequency ⟶ always pick this up.



Implementation

```
while(tt--) {
    int n, sum = 0; cin >> n;
    vector<int>types(n + 1), freq(n + 1);

    for(int i = 0; i < n; i++) {
        int x; cin >> x; types[x]++;
    }

    sort(types.begin(), types.end(), greater());

    for(int i = 0; i <= n; i++) {
        if(i) types[i] = max(0, min(types[i], types[i - 1] - 1));
        sum += types[i];
    }

    cout << sum << "\n";
}
```

type → frequency

Problem 2: Increasing Subsequence

Implement!

Problem Link: https://codeforces.com/problemset/problem/1157/C2

You are given a sequence a consisting of n integers.

You are making a sequence of moves. During each move you must take either the leftmost element of the sequence or the rightmost element of the sequence, write it down and remove it from the sequence. Your task is to write down a **strictly** increasing sequence, and among all such sequences you should take the longest (the length of the sequence is the number of elements in it).

For example, for the sequence $[1, 2, 4, 3, 2]$ the answer is 4 (you take 1 and the sequence becomes $[2, 4, 3, 2]$, then you take the rightmost element 2 and the sequence becomes $[2, 4, 3]$, then you take 3 and the sequence becomes $[2, 4]$ and then you take 4 and the sequence becomes $[2]$, the obtained increasing sequence is $[1, 2, 3, 4]$).

Problem 2: Increasing Subsequence

Implement!

1700

Examples

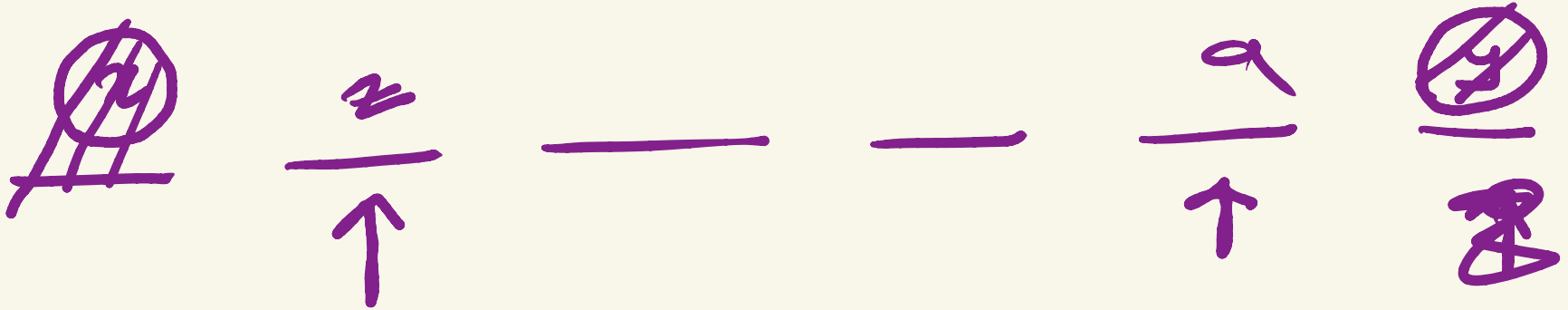
input	Copy
5 1 2 4 3 2	

output	Copy
4 LRRR	

input	Copy
7 1 3 5 6 5 4 2	

output	Copy
6 LRLRRR	

n



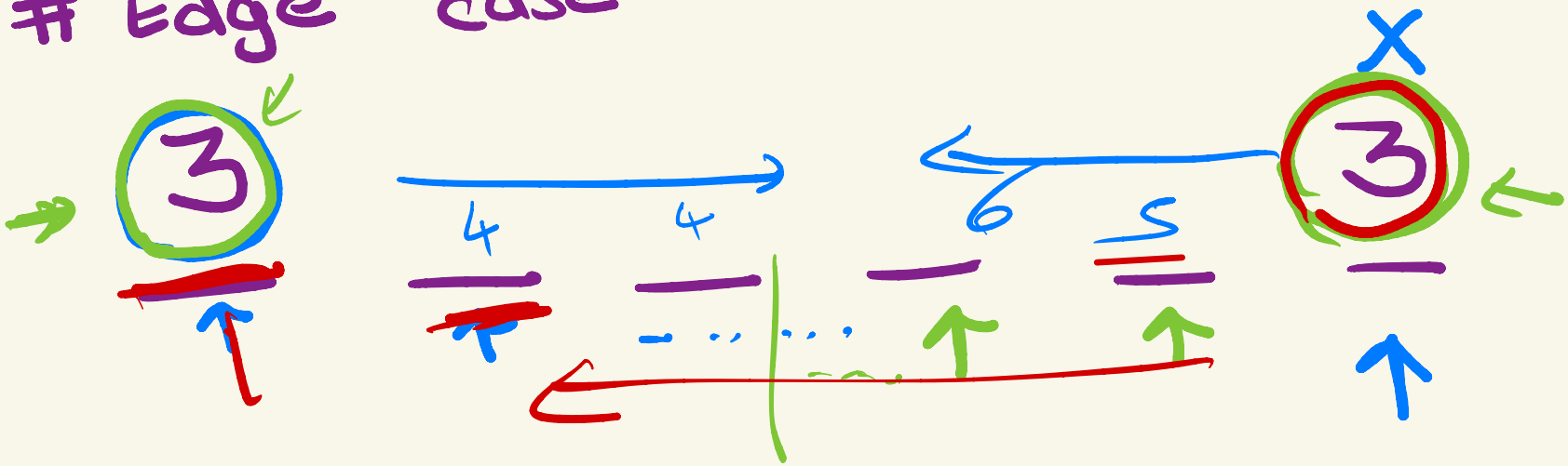
[x, y, z] ← Largest strictly increasing array

5 — — — = 3

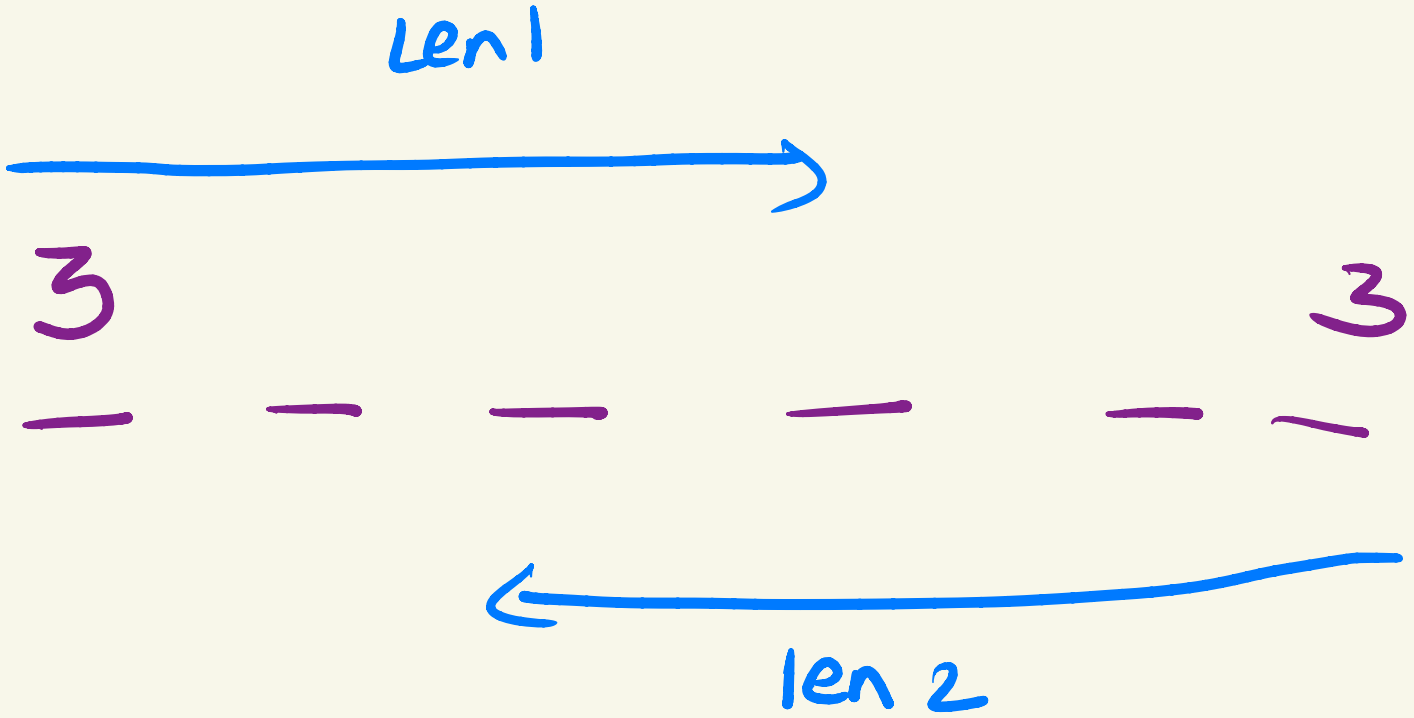
min out of the two

[3

Edge case



[3,



Implementation

```
int left = 0, right = n - 1, past = 0;
string answer = "";

while(left <= right) {
    if(a[left] < a[right]) {
        if(a[left] > past) past = a[left], answer += "L", left++;
        else if(a[right] > past) past = a[right], answer += "R", right--;
        else break;
    }
    else if(a[left] > a[right]) {
        if(a[right] > past) past = a[right], answer += "R", right--;
        else if(a[left] > past) past = a[left], answer += "L", left++;
        else break;
    }
    else if(a[left] == a[right]) {
        if(a[left] <= past) break;
        int newLeft = left + 1, newRight = right - 1;
        while(newLeft <= right && a[newLeft] > a[newLeft - 1]) newLeft++;
        while(newRight >= left && a[newRight] > a[newRight + 1]) newRight--;
        if((newLeft - left) >= (right - newRight)) answer += string((newLeft - left), 'L');
        else answer += string((right - newRight), 'R');
        break;
    }
}

cout << answer.size() << "\n" << answer << "\n";
```

Problem 3: Triangle Coloring

Greedy + Combinatorics

Problem Link: https://codeforces.com/problemset/problem/1795/D

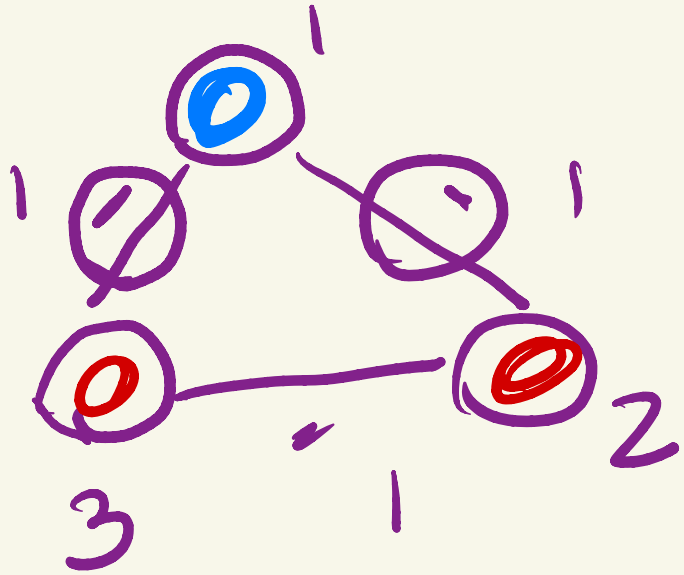
You are given an undirected graph consisting of n vertices and n edges, where n is divisible by 6. Each edge has a weight, which is a positive (greater than zero) integer.

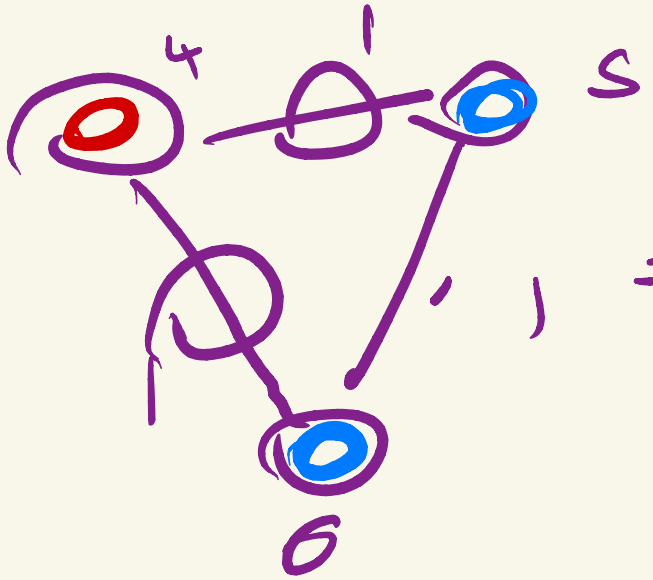
The graph has the following structure: it is split into $\frac{n}{3}$ triples of vertices, the first triple consisting of vertices $1, 2, 3$, the second triple consisting of vertices $4, 5, 6$, and so on. Every pair of vertices from the same triple is connected by an edge. There are no edges between vertices from different triples.

You have to paint the vertices of this graph into two colors, red and blue. Each vertex should have exactly one color, there should be exactly $\frac{n}{2}$ red vertices and $\frac{n}{2}$ blue vertices. The coloring is called valid if it meets these constraints.

The weight of the coloring is the sum of weights of edges connecting two vertices with different colors.

Let W be the maximum possible weight of a valid coloring. Calculate the number of valid colorings with weight W, and print it modulo 998244353.





= Cost of the graph

n/2 red

n/2 blue

number of ways

↳ max cost

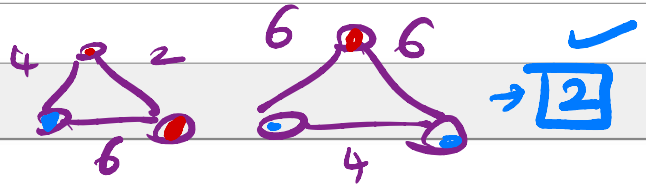
Problem 3: Triangle Coloring

Examples

input	Copy
12 1 3 3 7 8 5 2 2 2 2 4 2	

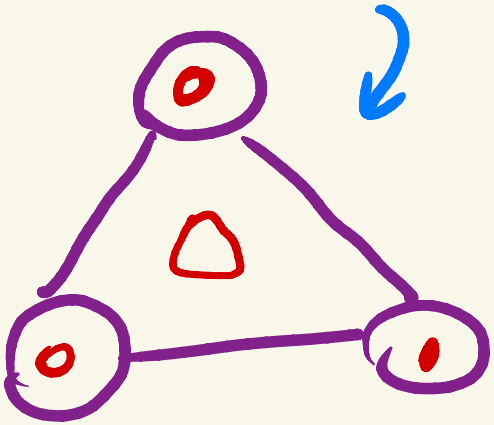
output	Copy
36	

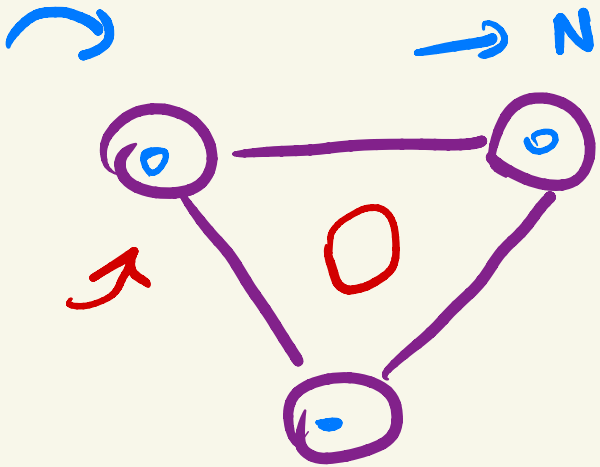
input	Copy
6 4 2 6 6 6 4	



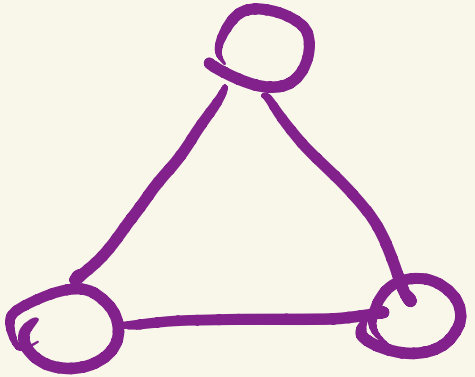
output	Copy
2	

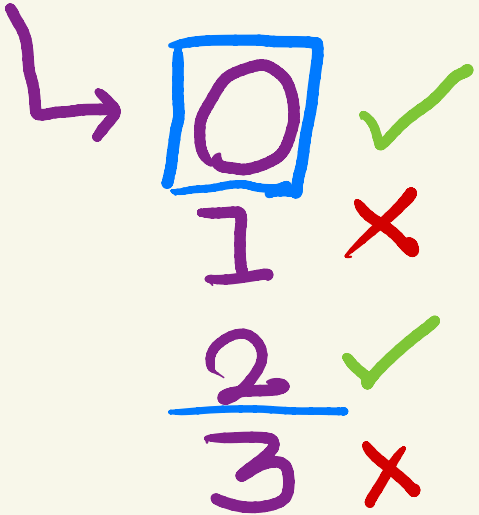
Note



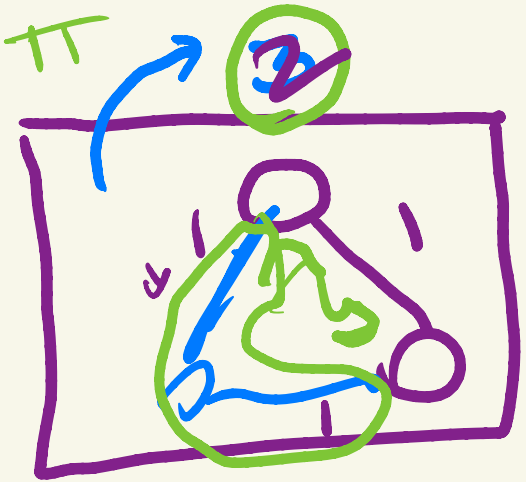


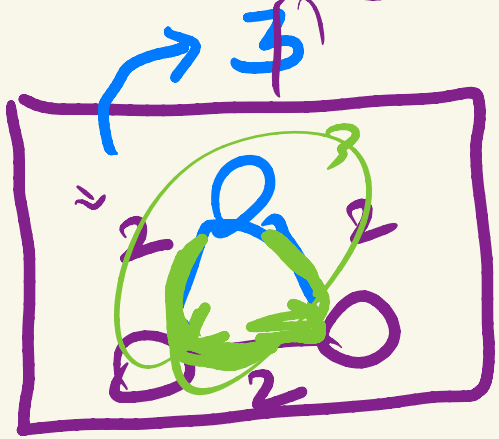
None are selected



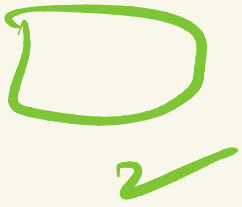


From each triad, we would select 2 edges exactly





↑ max sum

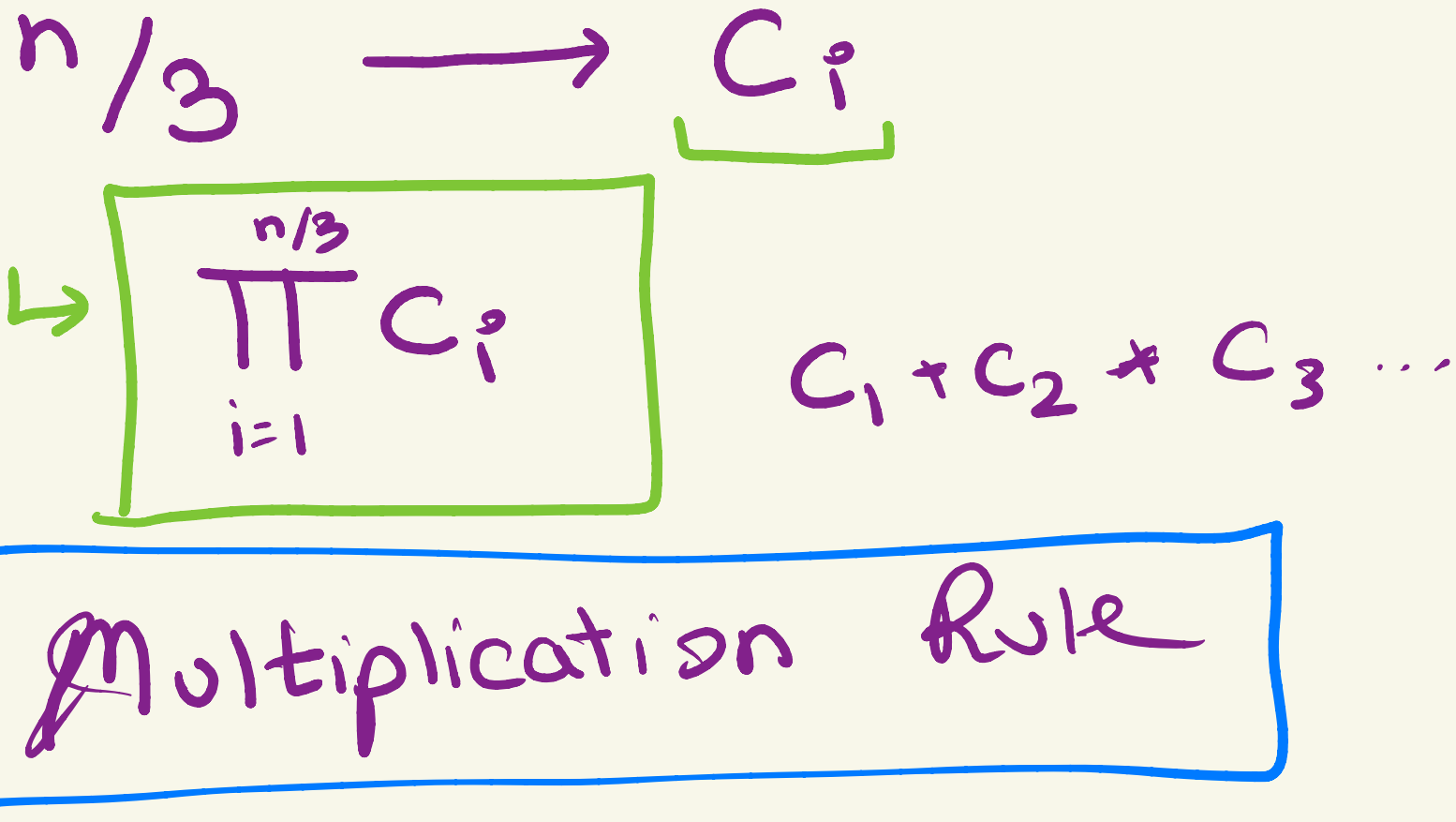


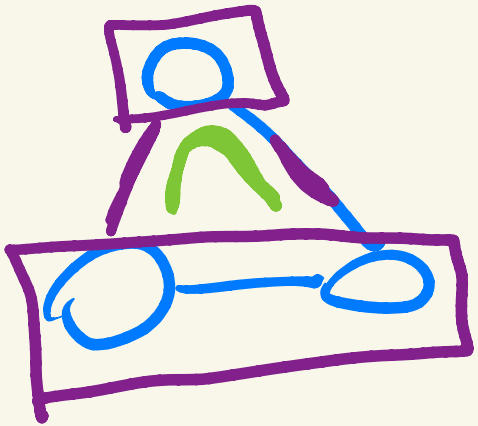
✓ 3 * 3 ?

2 * 3

~~3 * 3~~ ?

2 * 1



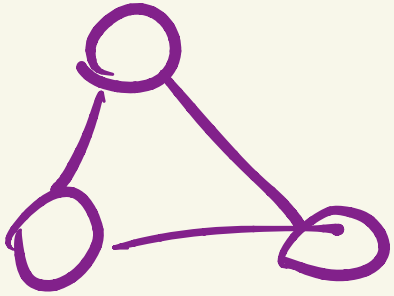
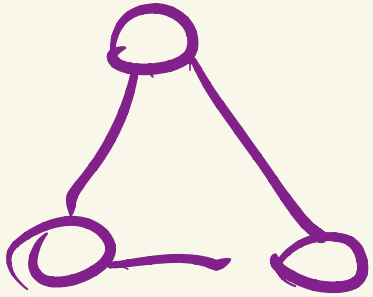
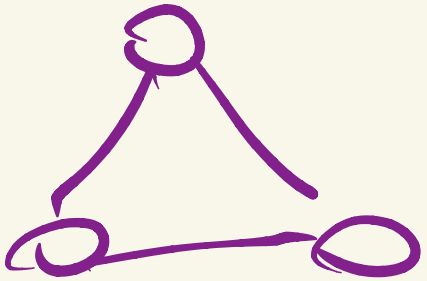
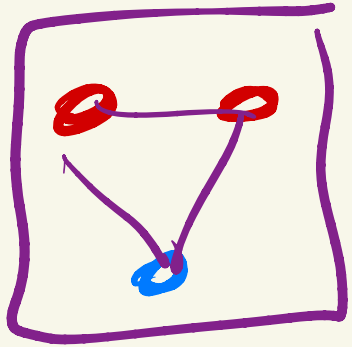
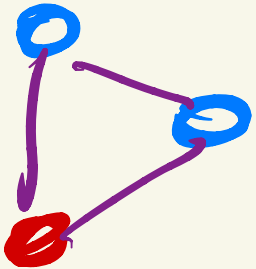
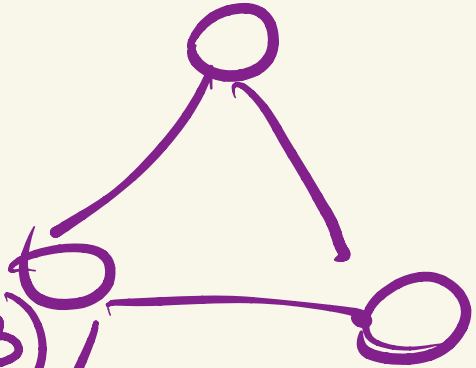


$n/3$

2 blue
1 red

2 red
1 blue

$$\checkmark \; {}^{n/3}C_{n/6} \cdot \prod_{i=1}^{n/3} \text{Choice}_i$$

$$^{n/3}C_{(n/3)/2} \longrightarrow {}^{n/3}C_{n/6}$$

arrangements of a component

$2 \times 3 \times 1 = 2$

```
for (1 — 2) {
    for (1 — 3) {
        for (1) {
            for (2) {
            }
        }
    }
}
```

Implementation

```
vector<int>a(n);
for(auto &i : a) cin >> i;

for(int i = 0; i < n; i+= 3)
{
    int mxWeight = max({a[i] + a[i + 1], a[i + 1] + a[i + 2], a[i] + a[i + 2]});
    int count = 0;
    if(a[i] + a[i + 1] == mxWeight) count++;
    if(a[i] + a[i + 2] == mxWeight) count++;
    if(a[i + 1] + a[i + 2] == mxWeight) count++;
    answer = (answer * count) % MOD;
}

cout << (answer * nCrModPFermat(n / 3, n / 6, MOD)) % MOD;
```

prod n/3 C n/6